

SAMAS gap analysis: six critical unknowns resolved

The previous SAMAS report selected the right components but left implementation details unresolved. This report fills those gaps with reverse-engineered architecture from Bhanu Teja P's Mission Control system, production-tested design patterns for 20+ agent orchestration, cost optimization strategies that reduce LLM spend by 80–95%, and consensus mechanisms backed by ACL/NeurIPS 2025 research. Each gap below delivers actionable blueprints rather than theoretical guidance — specific code patterns, proven UI component stacks, phased rollout timelines, and real-world cost numbers.

GAP 1: How Mission Control actually works under the hood

Bhanu Teja P (@pbteja1998) publicly detailed his system in a viral X thread (Dan-malone) (3.7M views, (X) January 31, 2026) before the YouTube video. The architecture is simpler than expected — and that simplicity is the insight.

One OpenClaw gateway, ten sessions. All agents run within a **single OpenClaw gateway daemon** (DeepWiki) on one VPS. Each agent is not a separate process but a separate *session* within the same gateway. Bhanu confirmed directly: "Ten agents equals ten sessions. Each waking up on their own schedule. Each with their own context." (X) Each session has a unique key (e.g., (agent:product-analyst:main)), a (SOUL.md) personality file, and JSONL conversation history persisted to disk. (X) The resource footprint per agent is minimal because **heavy inference happens on Anthropic's servers** — the local resource is just config files and temporary process execution during cron ticks. The original system had **10 agents** (MCU-themed: Jarvis, Shuri, Vision, Fury, Loki, Pepper, Wong, Friday, plus others), (Dan-malone) with additional agents like Hawkeye spawned dynamically later, likely reaching ~14 by the YouTube recording. (X)

The Mission Control Dashboard is a custom React app backed by Convex, a real-time serverless database. (X) It is not Notion, not a Google Sheet, and not something Jarvis coded via browser automation. The dashboard features an **activity feed** (real-time stream of all agent events), a **task board** (Kanban: Inbox → Assigned → In Progress → Review → Done), **agent cards** showing per-agent status, and a document panel for deliverables. (X) Bhanu described the aesthetic as "intentionally warm and editorial — like a newspaper dashboard." Convex was chosen for its real-time change propagation, TypeScript-native API, and generous free tier. (X) Convex now officially features Mission Control on their ecosystem page at (convex.dev/claw). (convex) (Convex) Multiple open-source clones already exist, including (manish-raana/openclaw-mission-control) (Convex + React) and (alanxurox/mission-control) (bash + SQLite, zero dependencies).

The 15-minute polling loop uses OpenClaw's built-in cron system, not external crontab. Each agent has a staggered cron schedule so they don't all fire simultaneously. Each cron tick creates an **isolated session** (one-shot): the agent wakes, queries Convex for new tasks and @mentions, performs assigned work, then terminates. (X) Only Jarvis maintains a persistent interactive session connected to Telegram. This keeps costs down — most agents consume zero resources between cron ticks. (X)

The "Squad Chat" is the shared Convex activity feed, not Telegram, Slack, or any external tool. All agents read and write to the same Convex database. (X) When an agent wakes on its schedule, it queries for new tasks, @mentions, and activity, then posts its own updates. A custom notification daemon polls for @mentions and routes them. The "broadcast" system is simply **posting to a shared Convex table that all agents check on their next heartbeat**. There is no simultaneous push — it is eventual delivery through the polling pattern.

Bhanu's permission model uses three tiers: Intern (needs approval for most actions), Specialist (works independently in their domain), and Lead (full autonomy, can delegate). (X) This maps directly to SAMAS's planned authority levels.

What remains unconfirmed: exact VPS specs and provider, monthly API costs, specific Claude model versions per agent, and whether Bhanu open-sourced his own Mission Control code (the repos above are community clones).

GAP 2: Architecture patterns for 20+ agent orchestration

Hierarchical supervision prevents orchestrator bottleneck

The dominant production pattern for 20+ agents is **LangGraph's multi-level hierarchical supervisor**. The top-level orchestrator routes between domain-specific sub-teams, each with its own supervisor. (PyPI) Sub-agents are exposed as "tools" the supervisor can call. (LangChain)

```
python

research_team = create_supervisor(
    [research_agent, math_agent],
    model=model, supervisor_name="research_supervisor"
).compile(name="research_team")

top_supervisor = create_supervisor(
    [research_team, writing_team, ops_team],
    model=model, supervisor_name="top_level"
).compile(name="top_level")
```

LangGraph's (Send) API enables parallel fan-out across workers. (Langchain) For bottleneck mitigation: distribute into domain sub-teams (3–5 agents each), use NATS JetStream for async priority queuing, and implement load balancing based on queue depth. Configurable authority maps to **LangGraph's (interrupt()) mechanism**: (LangChain) supervised mode interrupts all outputs, semi-autonomous interrupts only high-risk tool calls, and fully autonomous runs with post-hoc monitoring.

Memory Curator pattern resolves conflicting agent knowledge

The state-of-the-art is **hybrid graph + vector memory** using Graphiti or Cognee to bridge Neo4j and Qdrant. The Memory Curator agent should receive all write requests, perform entity resolution and deduplication

against the knowledge graph, resolve conflicts using priority rules (recency, source authority, consensus), and run periodic consolidation. Research from the **2025 Multi-Agent Memory Survey** identifies three hosting patterns: orchestrator-level (blackboard pattern), external shared DB, and per-agent private memory. (Techrxiv) For 20+ agents, multi-resolution summaries — global summaries plus agent-specific fine-grained logs — prevent information overload while preserving detail. (Techrxiv)

Security requires the "Rule of Two"

Meta's October 2025 paper establishes that agents must satisfy no more than two of three properties in a session: access to private data, exposure to untrusted content, and external communication capability. If all three are needed, human-in-the-loop is mandatory. Implementation follows a **Coordinator + Guard dual-agent architecture**: a Coordinator pre-filters inputs (classifies, sanitizes, consults policy store), while a Guard post-validates outputs (enforces format rules, blocks residual risks). (arXiv) A 2025 cross-agent provenance framework achieved **94% prompt injection detection accuracy** with 96% task accuracy retention (arXiv) using provenance ledgers that track data origin and trust levels across agents. (Quantum Zeitgeist)

Evolution Agent uses GEPA for automated prompt improvement

GEPA (Genetic-Pareto Reflective Prompt Evolution), available as (dspy.GEPA), outperforms MIPROv2 by >10% while using **35× fewer rollouts** than reinforcement learning approaches. (arXiv) (arXiv) The loop works as follows: sample a candidate from the Pareto frontier, collect execution traces and feedback on a minibatch, use LLM reflection to propose improved instructions, evaluate and update the frontier. (DSPy) For SAMAS, the Evolution Agent should run GEPA optimization weekly against production execution traces, A/B test optimized prompts, and gate deployment behind the QA Gateway for human review.

Agent SOPs belong in YAML + Git with database-backed runtime

CrewAI's role/backstory/goal pattern (Crew AI) (Firecrawl) in YAML is the proven standard. Store SOP templates in Git (source of truth) with semantic versioning (major.minor.patch), use a database-backed registry for runtime resolution, (Vinish) and enable the Evolution Agent to propose changes through pull requests. The recommended SOP structure includes: id, name, role, goal, backstory, constraints, tools, authority_level, escalation_rules, performance_metrics, and optimization_score.

Agent lifecycle follows a state machine from template to archive

The research supports this state flow: **TEMPLATE** → **INITIALIZING** → **ACTIVE** → **(HIBERNATED | ERROR | DEGRADED)** → **RETIRING** → **ARCHIVED**. Dynamic agent creation is validated by the **AgentSpawn paper (February 2026)**, which achieves 34% higher completion rates through automatic memory transfer during spawning and adaptive spawning policies. (arXiv) For 20+ agents, most should be **event-triggered via NATS JetStream subscriptions**. Only 3–5 core agents (Orchestrator, Security, Recovery, QA Gateway) run always-on.

GAP 3: What successful MVPs actually shipped — and how to replicate it

Every viral AI agent project started embarrassingly small

AgentGPT shipped a browser-only demo (zero install, type a goal, watch it work) and hit 100K daily users in its first week (DataCamp) — then burned **\$2,000/day** in API costs. (DataCamp) **CrewAI** launched as a pip-installable Python library (GitHub) with sequential task execution (GitHub +2) and ~200 lines of code to define a "crew"; it now executes 10M+ agents/month (VentureBeat) with \$18M raised. (GlobeNewswire) **Dify** started as a visual app builder with prompt IDE and RAG pipeline behind (docker compose up -d); it reached 60K stars. (GitHub) **OpenHands** rode the Devin hype wave with a single Docker command ((docker run -e LLM_API_KEY ... -p 3000:3000)) (GitHub) and hit #1 trending on GitHub. **OpenClaw** grew from 9K to 157K stars in 60 days through demo virality, a skills marketplace, and influencer amplification. (Growth Foundry) (maestro)

The pattern is unambiguous: **ship 2–3 agents doing one impressive task, not 20+ agents doing everything.**

Time-to-first-value determines adoption

Project	Setup method	Time to working demo
AgentGPT	Open browser	<1 minute
Dify	(docker compose up -d)	<5 minutes
OpenHands	Single (docker run)	<5 minutes
CrewAI	(pip install crewai)	<10 minutes
SuperAGI	Docker Compose + config	15–30 minutes

Developers abandon during setup when: too many API keys are required upfront, Docker behaves inconsistently on Windows, documentation lags code, and build times exceed 5 minutes. The winning formula is **pre-built Docker images, sensible defaults, and a free/mock LLM option** (Ollama) for testing without API keys.

The recommended four-phase roadmap

Phase 0 — "Hello World" (weeks 1–2): Three hardcoded agents (Researcher, Analyst, Writer) doing a single impressive task. (docker compose up) → dashboard at localhost:3000. No configuration beyond one LLM API key. A GIF demo in the README showing agents collaborating. Target: <5 minutes from clone to working demo.

Phase 1 — "Configurable Crew" (weeks 3–6): YAML-based agent definitions, 5–7 pre-built templates, basic NATS messaging visible in the dashboard, LangGraph orchestration. Target: HN front page, 1K stars.

Phase 2 — "Extensible Platform" (weeks 7–12): Custom agent creation via API/YAML, plugin architecture for tools, memory layer (conversation context → Qdrant archival), 10–15 agents, REST API. Target: 5K stars,

first external contributors.

Phase 3 — "Production Ready" (months 3–6): Full 20+ agent roster with enable/disable toggles, enterprise monitoring/observability, NATS JetStream for reliable delivery, horizontal scaling documentation, cloud hosting option. Target: 10K+ stars, enterprise pilots.

Growth requires timing, demos, and education

CrewAI's founder João Moura attributed growth to "opinionated simplicity" and "doubling down on education." VentureBeat Every project that crossed 10K stars had: a visual demo showing agents doing something tangible, launch timing aligned with industry hype, simultaneous multi-platform launch (HN, X, Reddit, Discord, ProductHunt), and strong documentation. The critical anti-pattern to avoid: OpenClaw's "I ship code I don't read" philosophy led to 341 malicious skills (11.3% of its marketplace) three days after reaching 100K stars. maestro

GAP 4: The dashboard stack that production systems actually use

React Flow + shadcn/ui + Tremor is the optimal combination

Based on analysis of seven production agent dashboards (SuperAGI, Dify, n8n, LangSmith, Langfuse, Grafana, CrewAI Studio), the recommended stack is:

Layer	Library	Purpose
UI components	shadcn/ui	Accessible, Tailwind-based primitives
Charts	Tremor + Recharts	KPI cards, time series, cost tracking
Workflow canvas	React Flow (@xyflow/react)	Agent topology, message flow visualization
Layout algorithm	Dagre / ELK	Automatic hierarchical node positioning
State management	Zustand	Lightweight reactive state
Real-time	WebSocket / SSE	Live agent status, activity feeds
Drag and drop	dnd-kit	Kanban task board
Agent protocol	AG-UI (@ag-ui/client)	Standardized agent-frontend events
HITL components	CopilotKit	Approval workflows, shared state

Key UI patterns drawn from production implementations

Dify's workflow canvas is the closest analog to what SAMAS needs. It uses ~15 core node types on an

infinite drag-and-drop canvas ([Baytech Consulting](#)) ([Dify](#)) with a debugging tree view showing agent reasoning chains. ([Dify](#)) Its pattern of separate Chatbot, Agent, and Workflow app types ([Skypage](#)) is worth adopting. ([GitHub](#))

Langfuse's open-source trace visualization provides a tree/timeline toggle for execution traces, ([Langfuse](#)) custom dashboards with multi-level aggregations, and curated dashboard templates (latency, cost, usage). ([Langfuse](#)) This is the best model for agent execution monitoring.

The autonomy slider was coined by Andrej Karpathy in June 2025 ([Substack](#)) and appears in production at Cursor (Tab → Edit → Chat → Agent), Perplexity (Search → Research → Deep Research), ([Substack](#)) and Replit (Low/Medium/High/Max). ([Replit Docs](#)) For SAMAS, implement a per-agent slider with 3–5 discrete levels mapped to HITL trigger configurations.

Human-in-the-loop approval follows the Vercel AI SDK pattern for Next.js: add (`needsApproval: true`) to tool definitions, the SDK pauses execution and sends an (`approval-requested`) state to the client, and the frontend renders Approve/Edit/Deny buttons. Conditional approval (e.g., (`needsApproval: async (input) => input.amount > 1000`)) is supported natively.

Multi-agent chat should use agent-specific avatars with color badges, collapsible reasoning traces per message, delegation transitions ("Handing off to [Agent]..."), Slack-style threaded sub-conversations for specialist work, and real-time status indicators per active agent. The AG-UI protocol from CopilotKit standardizes lifecycle events (RUN_STARTED, RUN_FINISHED), text streaming, tool calls, state snapshots, ([CopilotKit](#)) and HITL approvals ([Microsoft Learn](#)) — implementing AG-UI compatibility provides interoperability with the emerging ecosystem.

n8n's debugging patterns — "pin data" to nodes and "re-run single steps" ([n8n](#)) — are excellent for agent development and should be replicated. Their execution history with per-step inspection ([Businesscompassllc](#)) is proven at scale.

Starter templates accelerate development

The ([next-shadcn-dashboard-starter](#)) ([GitHub: Kiranism](#)) provides Next.js 16 + shadcn/ui + TypeScript + Tailwind with analytics overview, data tables, RBAC navigation, ([GitHub](#)) and a drag-and-drop task board using dnd-kit + Zustand. This is the recommended starting point for SAMAS's dashboard, extended with React Flow for the topology view and Tremor for agent metrics.

GAP 5: Cutting 24/7 multi-agent costs from \$50K to \$500/month

The naive approach is financially unsustainable

Running 20 agents on GPT-4o continuously without optimization costs an estimated **\$10,000–\$50,000/month**. Real-world case studies confirm these numbers: an e-commerce brand's single support agent hit \$7,500/month from unoptimized prompts, ([AgentiveAIQ](#)) and two agents stuck in a compliment loop burned \$500 in API credits

over a weekend. (thinkpeak) The full optimization stack achieves **80–95% cost reduction**, bringing the same system to **\$500–\$2,000/month**.

Model routing delivers the largest single cost reduction

RouteLLM (open-source, by LMSYS) routes between "strong" and "weak" models using four trained classifier algorithms. Results: **85% cost savings on MT Bench, 45% on MMLU**, (LMSYS Org) achieving 95% of GPT-4 performance while using GPT-4 for only 14% of queries. (Maxim Articles) **LiteLLM** serves as the self-hosted gateway with routing strategies (least-busy, usage-based, latency-based), per-team budgets in YAML, automatic retries, and Redis-backed state. (TrueFoundry) **Martian** uses mechanistic interpretability to predict model performance per-query without running inference — their finance pipeline went from 5.98% success (GPT-4 only) to 35.99% (Martian router) because per-step quality compounds. (Withmartian)

The task-to-model mapping for SAMAS agents breaks down cleanly:

- **Cheap/local models** (GPT-4o Mini at \$0.15/M input, (Silicondata) Gemini 2.5 Flash at \$0.15/M, Llama 3.3 70B via OpenRouter at \$0.12/M): (DETECTX) text classification, template generation, data extraction, log parsing, FAQs, simple summarization
- **Mid-tier models** (Claude Sonnet 4 at \$3/M, (IntuitionLabs) GPT-4o at \$2.50–\$5/M): complex code, multi-step reasoning, nuanced content, RAG with complex documents
- **Frontier models** (Claude Opus 4 at \$15/M, (IntuitionLabs) GPT-5): only for novel problem-solving with high stakes, complex legal/financial analysis, critical multi-hop reasoning

A routing rule sending 80–90% of requests to cheap models yields **3–5× cost reduction** alone.

Multi-tier caching compounds savings dramatically

Anthropic's prompt caching reduces input costs by **90%** on cache hits (reads cost $0.1 \times$ base price). (Introl) With 20 agents sharing stable system prompts, the break-even is just 2 API calls. One user reported going from **\$720/month to \$72/month** purely from prompt caching. (Medium) OpenAI's automatic caching gives a 50% discount on prompts $\geq 1,024$ tokens with no code changes. (Prompt Hub)

Semantic caching via GPTCache intercepts identical or near-identical queries before they reach the LLM. Experiments show **61–69% cache hit rates** with $>97\%$ accuracy. Portkey reports $\sim 20\%$ hit rates in production with 99% accuracy. A new approach — **agentic plan caching** — caches structured execution plans from completed agent runs, reducing costs by **50.3%** and latency by 27.3% while maintaining 96.7% accuracy.

(arXiv) (arXiv)

The recommended multi-tier architecture: Request $\rightarrow$ Semantic cache (100% savings on hit) $\rightarrow$ Prompt/prefix cache (50–90% savings) $\rightarrow$ Full inference (no savings). (Introl)

Self-hosting is a trap for most deployments

A surprising finding: **APIs are 60–700× cheaper** than self-hosting for the same open-weight model. Llama 3.3 70B via OpenRouter costs \$0.12/M tokens; the same model on a Lambda Labs A100 costs $\sim$ \$43/day to serve

~1M tokens. (s-anand) Self-hosting only makes economic sense at **100M+ tokens/month** or when fine-tuned models aren't available via API. SAMAS should start with APIs through LiteLLM as a gateway and only self-host specific high-volume small models (7B–13B) for simple classification tasks.

Event-driven scheduling eliminates idle-agent cost

Most agents should hibernate between tasks using NATS JetStream subscriptions. When no matching messages arrive, the agent consumes zero LLM tokens. Only 3–5 core agents (Orchestrator, Security, Recovery, QA Gateway) need always-on status. Non-urgent work should use **batch APIs** (Anthropic and OpenAI offer 50% discounts on batch jobs). Bhanu's SiteGPT approach validates this: agents wake on 15-minute staggered cron cycles, do their work, then terminate. (X)

GAP 6: When agents disagree — voting beats debate for most tasks

The ACL 2025 evidence is decisive

Kaesberg et al. tested seven decision protocols across six datasets (arxiv) using Llama 3 (8B and 70B). (arXiv) The headline result: **voting protocols improve reasoning tasks by 13.2%** while **consensus protocols improve knowledge tasks by 2.8%**. (arxiv) But the more counterintuitive finding is that **more discussion rounds before voting actually reduces performance** (arxiv) because discussion causes answer convergence, destroying the diversity that makes voting work.

The NeurIPS 2025 spotlight paper "Debate or Vote" proved this theoretically: **majority voting alone accounts for most performance gains** typically attributed to multi-agent debate. (arXiv) Debate induces a martingale over agents' belief trajectories (OpenReview) — it does not improve expected correctness without specific interventions that bias updates toward correction. (OpenReview)

Two novel techniques from the ACL paper significantly improve outcomes:

- **All-Agents Drafting (AAD)**: each agent independently drafts an initial solution before any interaction, adding **+3.3%** (arxiv)
- **Collective Improvement (CI)**: structured iterative refinement preventing excessive communication, adding **+7.4%** (arxiv)

Confidence-weighted voting is the practical implementation standard

The **ReConcile framework** (Chen et al., ACL 2024) provides the clearest implementation pattern. Three diverse LLMs generate initial answers with confidence scores (ACL Anthology) (arXiv) via a calibration prompt ("On a scale of 0–100%, how confident are you?"). Confidence scores serve as voting weights: (OpenReview)
$$\text{final_answer} = \text{argmax}(\text{sum}(\text{confidence}_i \times \text{vote}_i))$$
. Results surpass single-agent baselines by **up to 11.4%** and outperform GPT-4 on three datasets. (arXiv) (arXiv) The critical finding: **diversity from different model architectures is essential** — homogeneous agents perform significantly worse. (Liner) (arXiv)

For SAMAS, track per-agent, per-domain historical accuracy using an exponential moving average, then multiply current confidence by historical accuracy weight for each vote. This creates a self-calibrating system where agents that are consistently right in a domain accumulate more influence over time.

Structured debate is worth the cost only for high-stakes decisions

The optimal debate configuration across studies is **2–3 rounds with 3–5 heterogeneous agents**. ReConcile consistently converges by round 3. [\(Liner\)](#) [\(OpenReview\)](#) The **A-HMAD framework** (2025) adds dynamic routing that activates different agent subsets based on query type, [\(Springer\)](#) achieving 4–6% higher accuracy with 30%+ fewer factual errors. [\(Techxplore\)](#) For SAMAS's 20+ agents, full-group debate is impractical — route decisions to relevant **3–5 agent subgroups** based on domain expertise.

The governance layer: Constitutional AI meets trust scoring

The **Governance-as-a-Service (GaaS)** framework (Gaurav et al., August 2025) provides the most complete multi-agent governance pattern. [\(arXiv\)](#) It treats agents as untrusted by default, [\(ResearchGate\)](#) [\(arXiv\)](#) with three enforcement modes: **coercive** (hard blocks on prohibited actions), **normative** (warnings that escalate as trust decreases), and **adaptive** (thresholds that adjust based on violation patterns). Each agent maintains a Trust Factor $TF \in [0,1]$, updated continuously using severity-aware penalization. Policies are specified as JSON rules with pattern matching, severity levels, and audit logging.

For SAMAS, implement an adaptive protocol selector:

- **Factual verification** → majority consensus (knowledge task)
- **Strategic reasoning** → confidence-weighted voting with AAD (reasoning task)
- **Cross-domain decisions** → hierarchical debate among domain leads
- **High-stakes decisions** → structured debate + mandatory human approval
- **Default** → fast confidence-weighted vote

Human escalation triggers must be explicit

Escalation to human should fire when: average confidence across agents falls below 40%, agents oscillate between positions (loop detection), no convergence after 3 rounds, the domain is flagged as high-stakes, or strong persistent disagreement exists among confident agents. The escalation package should include a summary of all agent positions, their confidence scores, the full debate history, and a recommended action — giving the human everything needed for a fast, informed decision. [\(Jetlink\)](#)

Conclusion: the critical path forward

The six gaps reveal a consistent theme — **production multi-agent systems succeed through orchestrated simplicity, not architectural complexity**. Bhanu's Mission Control runs 10+ agents on a single VPS using nothing more than staggered cron jobs and a shared Convex database. [\(Dan-malone\)](#) CrewAI reached 10M

agents/month ([VentureBeat](#)) starting from 200 lines of Python. The ACL 2025 research shows majority voting captures most decision-quality gains without expensive debate rounds. ([OpenReview](#)) ([arXiv](#))

SAMAS's critical path follows from these findings. **Start Phase 0 with exactly three agents behind** ([docker compose up](#)), demonstrating visible multi-agent collaboration on a single impressive task. Use LiteLLM as the routing gateway from day one — the cost difference between naive and optimized approaches is 10–50×. Implement confidence-weighted voting as the default consensus mechanism, ([ACL Anthology](#)) reserving structured debate for high-stakes cross-domain decisions only. Build the dashboard on React Flow + shadcn/ui + Tremor using the ([next-shadcn-dashboard-starter](#)) template, with Langfuse-style trace visualization and an autonomy slider per agent. Version all SOPs in YAML + Git with GEPA-powered optimization running weekly against production traces.

The technologies already selected (NATS, LangGraph, Qdrant, Neo4j, DSPy, MCP+A2A, Next.js) are validated by this research. What this gap analysis adds is the **implementation sequence and specific patterns** that prevent the project from shipping a complex architecture before demonstrating a compelling demo — the anti-pattern that killed AgentGPT's momentum ([DataCamp](#)) and forced AutoGen's painful rewrite. ([Microsoft](#))